

Club Platform

Builds und Installer

Detaillierte Anleitung · Deutsch
Version 0.6.0 · ABI V3 · Stand 21.09.2026

Bunker Development

Diese PDF wird aus der Markdown-Dokumentation erzeugt. Für das Kopieren längerer Befehle die verlinkte Markdown-Fassung verwenden; lange Codezeilen können im Druck umbrechen. Relative Verweise beziehen sich auf das öffentliche Doku-Repository.

[Markdown-Fassung im Doku-Repository öffnen](#)

Inhalt

1. Welche Version benötige ich?	2
2. Grundprinzip und Arbeitsverzeichnisse	2
3. Gemeinsame Voraussetzungen	3
4. macOS: Vorbereitung	3
5. Windows: Vorbereitung	5
6. Linux: Vorbereitung	6
7. Optionen, Archive und PostgreSQL	7
8. Portal und Server lokal testen	8
9. Lizenzen richtig erzeugen und testen	9
10. Abnahme eines neu erzeugten Pakets	10
11. Fehlerbilder und gezielte Behebung	11
12. Quellen und Pflege	11

Detaillierte Arbeitsanleitung für Bunker Development · Deutsch · Stand 21.09.2026

Quellstand: Club Platform 0.6.0, Datenbankschema 18, Commit

[5679c9fd868559e12b2140d0da136e2a79879a3b](#) im Entwicklungszweig [feature/calendar-events](#). Diese Anleitung beschreibt diesen Entwicklungsstand, keine bereits abgenommene Produktfreigabe. Die Befehle wurden mit den vorhandenen Buildskripten und CMake-Dateien abgeglichen. Der macOS-Publisher-Installer wurde vom Betreiber erfolgreich installiert; daraus folgt keine Abnahme aller anderen Pakete.

PDF: Builds und Installer. Ergänzend: Module ohne Core-Quellcode, Aktivierungsdienst.

1. Welche Version benötige ich?

Variante	Inhalt und Verwendung	Buildprofil
Desktop, lokal	Qt-App, lokale SQLite-Datenbank, Fachmodule; Serverprogramm ebenfalls enthalten	desktop
Desktop als Serverclient	Dieselbe Desktop-App, Verbindung zum Club-Server; keine eigene lokale Fachdatenbank	desktop
Server	HTTP-/REST-Server und Fachmodule, keine Desktop-Oberfläche	server
Browserportal	Statische Weboberfläche; benötigt einen laufenden Club-Server	Zusatz <code>--with-portal</code>
Publisher / Lizenztool	Separate Qt-Widgets-App zum Ausstellen signierter Lizenzen	publisher
Aktivierungsdienst	Separater Python-Dienst für Aktivierung und Erneuerung	Kein natives Buildprofil

Editionen wie [sports-school](#) und freigeschaltete Fachmodule stehen in der Lizenz. Dafür existieren keine eigenen Buildprofile. Debug und Release sind Buildkonfigurationen. Standalone und aktivierungspflichtig sind Lizenzrichtlinien, keine separaten Desktopprogramme.

Ein Publisher-Installer installiert nicht die Kunden-Desktop-App. Für beide Programme sind zwei Pakete erforderlich. Ein Desktop-Installer enthält keinen privaten Herausgeberschlüssel und kein Publisher-Tool. Ein Server-Installer richtet noch keinen automatisch startenden Betriebssystemdienst ein.

2. Grundprinzip und Arbeitsverzeichnisse

Baue auf dem jeweiligen Zielsystem: Windows-Pakete auf Windows, macOS-Pakete auf macOS, Linux-Pakete auf Linux. Qt, Compiler und Bibliotheken müssen dieselbe Zielarchitektur verwenden. Apple Silicon und Intel benötigen jeweils einen passenden Build; ein Universal-Binary oder Cross-Compiler wird nicht automatisch erzeugt.

Die folgenden Befehle beginnen im Stammverzeichnis des privaten Repositorys [club-platform](#). Nur interne Entwickler mit Quellcodezugriff können diese Produktbuilds erstellen. Externe Modulentwickler verwenden ausschließlich das öffentliche SDK und die separate Modulanleitung.

Bei einem neuen Checkout:

```
git clone --branch feature/calendar-events \
  https://github.com/jborkenhagen74/club-platform.git
cd club-platform
git status --short
git log -1 --oneline
```

Unter PowerShell den Clone-Befehl in eine Zeile schreiben. Bei einem vorhandenen Checkout zuerst ungesicherte Änderungen prüfen und den gewünschten Quellstand bewusst auswählen. Nach Integration der Änderungen kann ein anderer Branch verwendet werden, sofern er die hier beschriebenen Änderungen enthält. Ein unverändert alter [develop](#)-Build genügt nicht.

Die Skripte bestimmen den Repositorypfad selbst. Für verständliche relative Schlüsselpfade und die unten gezeigten Startbefehle trotzdem im Repository bleiben.

Artefakt	Ort
Native Buildbäume	build/<system>-<profil>-<konfiguration>
Zusätzlicher Testbuild bei festem Public-Key	Derselbe Name mit <code>-verification</code>
Fertige Installer und Archive	out/installers
CPack-Zwischenverzeichnis	out/installers/_CPack_Packages

`_CPack_Packages` allein ist kein fertiger Installer. Maßgeblich sind ein erfolgreicher Prozessabschluss und eine fertige `.pkg`, `.exe`, `.deb`, `.rpm`, `.zip` oder `.tar.gz`-Datei.

3. Gemeinsame Voraussetzungen

Komponente	Anforderung
Python	3.9 oder neuer für <code>scripts/build.py</code>
CMake / CTest / CPack	Projektminimum 3.25; für Generator Visual Studio 18 2026 mindestens CMake 4.2
Compiler	C++23-fähig; passende Standardbibliothek
Qt	Mindestens 6.5; im Projekt verwendetes Referenz-SDK 6.11.2
Desktop-Qt	Core, Gui, Qml, Quick, QuickControls2, Concurrent, Network
Publisher-Qt	Core, Gui, Widgets aus Qt Base
Native Bibliotheken	SQLite, ICU, libxml2, curl, libsodium, cpp-httpplib, nlohmann-json
Portal optional	Node.js 22 gemäß Portal-CI; passende aktuelle 22er-Version für das Vite-7-Lockfile und npm

Auch beim Publisher konfiguriert CMake Teile des Gesamtprojekts. Deshalb die nativen Projektabhängigkeiten vollständig installieren. Serverbuilds benötigen keine Qt-Oberfläche. Die Skripte installieren keine Compiler und keine Systempakete.

Vor einem langen Build prüfen:

```
python3 --version
cmake --version
ctest --version
cpack --version
python3 scripts/build.py --help
```

Unter Windows `python3` durch `py -3` ersetzen. Bei einer restriktiven PowerShell-Skriptrichtlinie direkt den Python-Treiber verwenden; eine globale Lockerung der Ausführungsrichtlinie ist nicht erforderlich.

4. macOS: Vorbereitung

1. Xcode bzw. die Command Line Tools installieren und deren Erstinitialisierung abschließen. Compiler und Paketwerkzeuge prüfen.

2. Die Homebrew-Abhängigkeiten installieren. Die Befehle setzen eine bereits vorhandene Homebrew-Installation voraus.
3. Im Qt-Installer das macOS-SDK einschließlich Qt Quick für Desktopbuilds installieren. Den tatsächlichen Installationspfad verwenden.

```
xcode-select -p
clang++ --version
xcrun --find pkgbuild
xcrun --find productbuild
brew install cmake ninja python sqlite libxml2 icu4c curl \
  libsodium cpp-httplib nlohmann-json pkgconf
export QT_ROOT="$HOME/Qt/6.11.2/macos"
export ICU_ROOT="$(brew --prefix icu4c)"
test -d "$QT_ROOT/lib/cmake/Qt6" || echo "QT_ROOT korrigieren"
```

Wenn die letzte Prüfung fehlschlägt, zuerst `QT_ROOT` auf Dein tatsächlich installiertes SDK ändern. Auf Apple Silicon keine Intel-Homebrew-Bibliotheken mit einem ARM-Build mischen. `uname -m` zeigt die Architektur des laufenden Systems; die konkrete Compilerkonfiguration muss ebenfalls dazu passen.

4.1 Publisher bauen und starten

```
bash scripts/build-publisher.sh --config Release --generator Ninja
open build/macos-publisher-release/tools/publisher/clubplatform-license-tool.app
```

Im Tool unter **Einrichtung** einen vorhandenen Herausgeberschlüssel auswählen oder einmalig ein neues Schlüsselpaar erzeugen. Das gewählte Verzeichnis enthält `secret.hex` und `public.hex`. Den privaten Schlüssel sicher aufbewahren. Für bestehende Kunden keine neuen Schlüssel bei jedem Build erzeugen.

4.2 Publisher-Installer erstellen

```
bash scripts/create-publisher-installer.sh --generator Ninja
open out/installers
```

Im Finder die neu erstellte `.pkg` mit `publisher` im Namen öffnen. Beispiel auf Apple Silicon: `club-platform-publisher-0.6.0-macos-arm64.pkg`. Nach Abschluss starten:

```
open /Applications/ClubPlatformPublisher/clubplatform-license-tool.app
```

Der Build verwendet heute direkt `pkgbuild` und `productbuild`. Alte Fehler zu `CPACK_RESOURCE_FILE_README` oder unerlaubten Lizenzdateiendungen deuten auf einen älteren Verpackungsablauf hin. Nicht eine temporäre `CPackConfig.cmake` von Hand reparieren, sondern den aktuellen Quellstand bauen.

4.3 Öffentlichen Kundenschlüssel auswählen

Der folgende Dialog erwartet den echten Pfad zur `public.hex` des in der Publisher-App ausgewählten Herausgebers. Keine Beispielpfade wie `/vollständiger/Pfad/...` übernehmen.

```
printf 'Pfad zur vorhandenen issuer/public.hex: '
IFS= read -r ISSUER_PUBLIC
test -f "$ISSUER_PUBLIC" || echo "Datei fehlt: Pfad korrigieren"
```

Nur fortfahren, wenn die Datei existiert. Die Datei enthält den öffentlichen 64-stelligen Hex-Schlüssel. `secret.hex` enthält privaten Schlüsselstoff und darf niemals als Buildargument oder Kundenbestandteil verwendet werden.

4.4 Desktop- und Server-Installer

```
bash scripts/create-installer.sh --profile desktop \
  --public-key "$ISSUER_PUBLIC" --generator Ninja
open out/installers
```

Das Paket mit `desktop` im Namen installieren. Die laufende alte App vorher mit **Cmd+Q** beenden. Anschließend ausdrücklich die installierte App öffnen:

```
open /Applications/ClubPlatform/clubplatform-desktop.app
```

Separater Server-Installer:

```
bash scripts/create-installer.sh --profile server \
  --public-key "$ISSUER_PUBLIC" --generator Ninja
```

Desktop und Server verwenden auf macOS denselben Kunden-Installationsstamm [/Applications/ClubPlatform](#). Die Varianten daher auf getrennten Testsystemen oder kontrolliert nacheinander prüfen; sie sind keine automatisch isolierten Parallelprodukte.

Nur einen Desktop-Debugbuild für Entwicklung erzeugen:

```
bash scripts/build.sh --profile desktop --config Debug --generator Ninja
open build/macos-desktop-debug/apps/desktop/clubplatform-desktop.app
```

5. Windows: Vorbereitung

1. Visual Studio 2026 mit **Desktopentwicklung mit C++**, Windows-SDK und dem zum Qt-SDK passenden MSVC-v143-Toolset installieren.
2. CMake ab 4.2, Git und Python 3 installieren; [cmake](#), [ctest](#), [cpack](#) und [py](#) müssen erreichbar sein.
3. Qt 6.11.2 für [msvc2022_64](#) installieren. Nicht das MinGW-SDK mit MSVC mischen.
4. vcpkg außerhalb des Produktrepositorys bereitstellen und [VCPKG_ROOT](#) setzen. Die Manifestabhängigkeiten kommen aus [vcpkg.json](#); dessen Baseline nicht beiläufig aktualisieren.
5. Für native [.exe](#)-Installer NSIS ab 3.03 installieren; [makensis](#) muss auffindbar sein.

Beispiel für eine neue vcpkg-Installation; nur ausführen, wenn dort noch kein Checkout liegt:

```
git clone https://github.com/microsoft/vcpkg.git C:\Dev\vcpkg
& C:\Dev\vcpkg\bootstrap-vcpkg.bat
$env:VCPKG_ROOT = 'C:\Dev\vcpkg'
$env:QT_ROOT = 'C:\Qt\6.11.2\msvc2022_64'
cmake --version
py -3 --version
Test-Path "$env:QT_ROOT\lib\cmake\Qt6"
```

Pfade an die vorhandene Installation anpassen. Im privaten Produktrepository eine Developer PowerShell für Visual Studio öffnen. Ohne explizite Toolchaindatei übernimmt das Projekt aus [VCPKG_ROOT](#) die vcpkg-Toolchain und den Triplet [x64-windows-static-md](#). Abweichende Architekturen benötigen eine konsistente eigene Toolchain/Qt-Konfiguration.

5.1 Publisher bauen und installieren

```
py -3 scripts/build.py --profile publisher --config Release `
  --generator 'Visual Studio 18 2026'
& .\build\windows-publisher-release\tools\publisher\Release\clubplatform-license-tool.exe
```

Im Tool den vorhandenen Herausgeberschlüssel auswählen oder einmalig erzeugen. Danach Installer erstellen:

```
py -3 scripts/build.py --profile publisher --config Release `
  --installer --generator 'Visual Studio 18 2026'
Invoke-Item .\out\installers
```

Die [.exe](#) mit [publisher](#) im Namen ausführen. Standardmäßig verwendet das Paket ein separates [ClubPlatformPublisher](#)-Verzeichnis unter Programme und einen Startmenüeintrag.

5.2 Kundenschlüssel und Desktop-/Serverpakete

```
$issuerPublic = Read-Host 'Echter Pfad zur issuer/public.hex'
if (-not (Test-Path -LiteralPath $issuerPublic -PathType Leaf)) {
  throw 'Public-Key-Datei nicht gefunden'
}
py -3 scripts/build.py --profile desktop --config Release `
  --installer --public-key "$issuerPublic" `
  --generator 'Visual Studio 18 2026'
Invoke-Item .\out\installers
```

Den Desktop-Installer ausführen und die App über das Startmenü öffnen. [publisher](#) und [desktop](#) sind separate Downloads. Das Serverpaket wird so gebaut:

```
py -3 scripts/build.py --profile server --config Release `
  --installer --public-key "$issuerPublic" `
  --generator 'Visual Studio 18 2026'
```

Desktop und Server verwenden standardmäßig den Kundenstamm [ClubPlatform](#). Keine unabhängige Installation beider Varianten im selben Verzeichnis voraussetzen.

5.3 IncrediBuild optional

Die vorhandenen Skripte sind ein zusätzlicher Entwicklungsweg. Voraussetzung ist eine eingerichtete IncrediBuild-Installation mit [BuildConsole](#) ab 10.31 und passendem Visual-Studio-Toolset.

```
.\scripts\build-incredibuild.ps1 -Configuration Release -Desktop
```

Zusatzoptionen sind [-Rebuild](#), [-NoMonitor](#) und [-SkipTests](#). Dieser Weg erzeugt nicht automatisch einen Installer. Die normalen Build-/Installerwrapper aktivieren IncrediBuild nicht. Die Presets verwenden andere Buildverzeichnisse; für die Paketierung die oben beschriebenen Installerbefehle ausführen und nicht stillschweigend Artefakte verschiedener Buildbäume vermischen.

6. Linux: Vorbereitung

Die Paketnamen im folgenden Beispiel gelten für Debian/Ubuntu. Für andere Distributionen die entsprechenden Entwicklungspakete verwenden. Ein ausreichend neuer C++23-Compiler und ein passendes Qt-SDK müssen verfügbar sein.

```
sudo apt update
sudo apt install build-essential git cmake ninja-build python3 \
  pkg-config dpkg-dev libxml2-dev libicu-dev \
  libcurl4-openssl-dev libsqlite3-dev libsodium-dev \
  libcpp-http-lib-dev nlohmann-json3-dev
```

Falls das Distributions-CMake das Projektminimum unterschreitet, vor dem Build eine passende CMake-Version bereitstellen. Qt 6.11.2 einschließlich Qt Quick für Desktop installieren; beim Server kann Qt entfallen.

```
export QT_ROOT="$HOME/Qt/6.11.2/gcc_64"
test -d "$QT_ROOT/lib/cmake/Qt6" || echo "QT_ROOT korrigieren"
g++ --version
cmake --version
```

[gcc_64](#) ist das Beispiel für ein x86_64-Qt-SDK, kein ARM-SDK. Distribution, glibc und Zielarchitektur bestimmen die Binärkompatibilität. Auf einer geeigneten ältesten unterstützten Zielumgebung bauen und auf jeder angebotenen Distribution testen.

6.1 Publisher

```
bash scripts/build-publisher.sh --config Release --generator Ninja
./build/linux-publisher-release/tools/publisher/clubplatform-license-tool
bash scripts/create-publisher-installer.sh --generator Ninja
```

Für die Installation den tatsächlich erzeugten Paketnamen auswählen, besonders wenn mehrere Versionen im Ausgabeordner liegen:

```
printf 'Pfad zur erzeugten Publisher-DEB-Datei: '
IFS= read -r INSTALLER
sudo apt install "$(realpath "$INSTALLER")"
/opt/club-platform-publisher/bin/clubplatform-license-tool
```

6.2 Desktop und Server

Zuerst den existierenden öffentlichen Herausgeberschlüssel auswählen:

```
printf 'Pfad zur vorhandenen issuer/public.hex: '
IFS= read -r ISSUER_PUBLIC
test -f "$ISSUER_PUBLIC" || echo "Datei fehlt: Pfad korrigieren"
bash scripts/create-installer.sh --profile desktop \
  --public-key "$ISSUER_PUBLIC" --generator Ninja
```

Die neu erzeugte Desktop-DEB-Datei wie oben über `sudo apt install` installieren. Start:

```
/opt/club-platform-desktop/bin/clubplatform-desktop
```

Serverpaket:

```
bash scripts/create-installer.sh --profile server \
  --public-key "$ISSUER_PUBLIC" --generator Ninja
```

Installiertes Serverprogramm: `/opt/club-platform-server/bin/clubplatform-server`. Für RPM zusätzlich `rpmbuild` auf dem Buildhost bereitstellen und `--format RPM` an den jeweiligen Installerbefehl anhängen. Ein auf Debian gebautes RPM ist dadurch nicht automatisch für jede RPM-Distribution kompatibel; Zielsystemtests und deren Abhängigkeiten bleiben erforderlich.

7. Optionen, Archive und PostgreSQL

Option	Wirkung
<code>--config Debug</code>	Entwicklungsbuid, Diagnosen aktiviert; Standard ohne Installer
<code>--config Release</code>	Optimierter Build; Pflicht für Installer
<code>--config RelWithDebInfo</code>	Optimierter Build mit Debuginformationen, kein Installerprofil
<code>--jobs 4</code>	Parallelität begrenzen; Standard höchstens acht Jobs
<code>--public-key DATEI</code>	Öffentlichen Lizenzvertrauensanker fest einbauen
<code>--qt-root PFAD</code>	Qt-Pfad statt Umgebungsvariable <code>QT_ROOT</code>
<code>--toolchain DATEI</code>	Explizite CMake-Toolchain
<code>--preset NAME</code>	Vorhandenes Configure-Preset als Grundlage
<code>--output PFAD</code>	Ziel für fertige Pakete statt <code>out/installers</code>
<code>--dry-run</code>	Befehle anzeigen; kein Build. Public-Key-Datei wird trotzdem geprüft
<code>--skip-tests</code>	Tests überspringen; keine gleichwertige Freigabeprüfung
<code>--with-portal</code>	Portal mit npm bauen und mit Kundenpaket ausliefern

ZIP und TGZ werden auf allen drei Systemen angeboten. Beispiel:

```
bash scripts/create-installer.sh --profile desktop \
  --public-key "$ISSUER_PUBLIC" --format ZIP
```

Unter Windows dieselben Argumente an `py -3 scripts/build.py --config Release --installer` übergeben. Archive sind Verzeichnisarchive ohne native Installation, Dienstregistrierung oder Garantie, sämtliche Systembibliotheken mitzubringen.

Ein vorhandenes Benutzerpreset kann Toolchain- und Abhängigkeitsoptionen bereitstellen. Der Python-Treiber setzt aber sein eigenes `-B-`Verzeichnis und überschreibt Profil-, Test-, GUI- und Lizenzoptionen. Tests deshalb gegen das tatsächlich ausgegebene Buildverzeichnis ausführen, nicht gegen den ursprünglichen Binärpfad des Presets.

PostgreSQL ist standardmäßig ausgeschaltet. Für einen Server mit PostgreSQL zusätzlich `libpq`-Entwicklungspakete bereitstellen und in einem eigenen `CMakeUserPresets.json`-Configure-Preset `CLUBPLATFORM_WITH_POSTGRESQL` auf `ON` setzen. Dieses Preset mit `--preset` auswählen. Der Python-Treiber akzeptiert keine frei angehängten `-D...`-Optionen. SQLite bleibt der unkomplizierte lokale Testweg.

7.1 Konkretes PostgreSQL-Preset

Beispiel für einen neuen Eintrag in `CMakeUserPresets.json` im privaten Repository. Existierende Benutzerpresets ergänzen, nicht überschreiben:

```
{
  "version": 6,
  "configurePresets": [{
    "name": "customer-postgres-release",
    "inherits": "local-release-minimal",
    "cacheVariables": {
      "CLUBPLATFORM_WITH_POSTGRESQL": "ON"
    }
  }]
}
```

Unter Debian/Ubuntu zusätzlich `libpq-dev` installieren. Unter macOS kann `brew install libpq` die Clientbibliothek bereitstellen; deren Prefix bei Bedarf im Preset als `PostgreSQL_ROOT` ergänzen. Unter Windows die PostgreSQL-Cliententwicklung mit zum Compiler passender Architektur bereitstellen und `PostgreSQL_ROOT` im Preset auf deren tatsächlichen Installationsstamm setzen. Clientbibliothek und PostgreSQL-Datenbankdienst sind getrennte Voraussetzungen.

macOS/Linux:

```
bash scripts/create-installer.sh --profile server \
  --preset customer-postgres-release \
  --public-key "$ISSUER_PUBLIC" --generator Ninja
```

Windows:

```
py -3 scripts/build.py --profile server --config Release --installer `
  --preset customer-postgres-release --public-key "$issuerPublic" `
  --generator 'Visual Studio 18 2026'
```

Beim Start statt `--sqlite` die Option `--postgres` verwenden und die Verbindung über `CLUBPLATFORM_POSTGRESQL` konfigurieren. Datenbank, Dienstkonto und Zugangsdaten müssen zuvor eingerichtet sein. Diese Geheimnisse gehören in die lokale Dienstkonfiguration, nicht in Presets, Installer oder Repository. SQLite und PostgreSQL sind alternative Datenbankoptionen desselben Servers, keine getrennten Oberflächen.

8. Portal und Server lokal testen

Das Portal ist ein Browserfrontend, keine zusätzliche native Desktopvariante. Mit `--with-portal` führt der Treiber im Portalprojekt `npm ci` und den Produktionsbuild aus. Das Ergebnis landet im Kundenpaket unter `share/clubplatform/portal`. Ein Webserver, HTTPS und die API-Verbindung werden dadurch noch nicht eingerichtet.

Beispiel mit einem bereits ausgewählten Public-Key:

```
bash scripts/create-installer.sh --profile server \
  --public-key "$ISSUER_PUBLIC" --with-portal
```

Für einen isolierten Server-Funktionstest ein neues, beschreibbares Datenverzeichnis verwenden. Beispiel Linux nach Installation des Serverpakets:

```
mkdir -p "$HOME/club-platform-test"
SERVER=/opt/club-platform-server/bin/clubplatform-server
"$SERVER" --sqlite "$HOME/club-platform-test/test.sqlite" --init admin
"$SERVER" --sqlite "$HOME/club-platform-test/test.sqlite" --port 8080
```

Der Initialisierungsbefehl fragt das Passwort ab und beendet sich. Danach den Server im Vordergrund laufen lassen. Unter macOS liegt die installierte Binärdatei unter `/Applications/ClubPlatform/bin/clubplatform-server`, unter Windows im `bin`-Verzeichnis der gewählten Installation. Dieselben Optionen gelten dort; PowerShell ruft einen in einer Variablen gespeicherten Programmpfad mit `& $server` auf.

Eine installierte Desktop-App kann mit `--server http://localhost:8080` an diesen lokalen Testserver angeschlossen werden. macOS-Beispiel in einem zweiten Terminal:

```
open -n /Applications/ClubPlatform/clubplatform-desktop.app \
--args --server http://localhost:8080
```

Für entfernte Server HTTPS verwenden. `--database` und `--server` dürfen nicht gemeinsam gesetzt werden. Lizenz und native Module werden im Serverbetrieb auf dem Server verwaltet, nicht auf jedem Remote-Desktop oder im Browser. Für ein getrennt gehostetes Portal den erlaubten Origin mit `--portal-origin` auf die konkrete Portaladresse konfigurieren; nicht pauschal beliebige Origins freigeben.

9. Lizenzen richtig erzeugen und testen

9.1 Drei getrennte Schlüsselrollen

Rolle	Privater Schlüssel	Öffentlicher Schlüssel
Herausgeber / issuer	Nur bei Bunker Development im Publisher	Fest im Kundenbuild über <code>--public-key</code>
Aktivierungsdienst	Ausschließlich auf dem Diensthost	Bestandteil der aktivierungspflichtigen Lizenz
Modulkatalog	Beim freigebenden Herausgeber	Separater Vertrauensanker für Paket-/Katalogprüfung

Ein neuer Herausgeberschlüssel passt nicht zu einem bereits mit einem anderen Public-Key gebauten Kundenprogramm. Eine Neuinstallation löst dieses Missverhältnis nur, wenn der neue Build den richtigen öffentlichen Schlüssel enthält. Schlüsselrotation ist ein eigener Migrationsvorgang.

9.2 Standalone: ohne Online-Aktivierung

1. Aktualisierten Publisher und aktualisierte Kunden-App aus diesem Quellstand bauen und installieren. Alte laufende Prozesse beenden.
2. Im Publisher unter **Einrichtung** den vorhandenen privaten Herausgeberschlüssel auswählen.
3. Unter **Lizenz** die Option **Ohne Online-Aktivierung (keine Installationsbindung)** einschalten.
4. Lizenz-ID, Edition, Benutzerlimit, Ablauf-/Wartungsdatum und Module festlegen. Modulabhängigkeiten berücksichtigen.
5. **Nur signierte Lizenzdatei erstellen** wählen und einen neuen Dateinamen verwenden.
6. Als Benutzer mit `schema.manage` in der Kunden-App die Datei unter **Erweiterungen → Lizenzdatei importieren** laden.
7. Die Lizenz wird lokal geprüft; **Installation aktivieren** ist für diesen Modus nicht erforderlich.

Die Datei enthält die signierte Richtlinie `activation_mode: "standalone"`. Sie ist weiterhin signiert und zeitlich sowie auf Benutzer und Module begrenzt. Sie erzwingt keine Installationsbindung und kein zentrales Installationslimit; eine Online-Sperrung ist nicht verfügbar. Eine alte Lizenz ohne Aktivierungsobjekt ist nicht automatisch eine gültige neue Standalone-Lizenz.

9.3 Aktivierungspflichtige Lizenz

1. Standalone-Option ausschalten.
2. Tatsächlich erreichbare HTTPS-Adresse des Aktivierungsdienstes und dessen öffentlichen Schlüssel konfigurieren. Testadressen sind keine funktionierenden Dienste.

3. Installationslimit und Erneuerungsfrist festlegen. **Dauerhafte Offline-Aktivierung erlauben** nur bewusst aktivieren.
4. Signierte Lizenz erzeugen und beim Dienst registrieren. Mit eingerichtetem SSH-Zugang übernimmt **Lizenz erstellen und beim Dienst registrieren** beide Schritte.
5. Erst nach erfolgreicher Registrierung ausliefern. Alternativ die erzeugte Datei über die Verwaltungs-CLI des Dienstes registrieren.
6. Kunde importiert die Lizenzdatei und wählt **Installation aktivieren**.

Die Aktivierung ist an die Installation gebunden. Der laufende Host prüft regelmäßig, ob eine Erneuerung fällig ist. Eine gültige Aktivierung übersteht vorübergehende Netzausfälle bis zum Ablauf ihrer Frist. Das ist keine zwingende Online-Abfrage bei jedem Programmstart.

Der Aktivierungsdienst ist ein separater Python-/WSGI-Dienst, nicht der Club-REST-Server. Für dessen Einrichtung, HTTPS-Betrieb, Registrierung, Widerruf und Offline-Antworten die vollständige Aktivierungsanleitung verwenden. Gunicorn ist dort für den Linux-Diensthost vorgesehen. Für einen dauerhaften Betrieb zusätzlich Dienstverwaltung, Backups und Reverse-Proxy einrichten.

Die Dienstumgebungsvariable `CLUB_LICENSE_PUBLIC_KEY` erwartet den Hex-Inhalt des öffentlichen Schlüssels; die Verwaltungsoption `--license-public-key` erwartet dagegen einen Dateipfad. Diese beiden Formen nicht vertauschen.

10. Abnahme eines neu erzeugten Pakets

1. Quellcommit, Qt-/Compiler-/CMake-Version, Betriebssystem und Architektur protokollieren. Den vollständigen Build erfolgreich abschließen lassen.
2. Fertiges Paket auf einem separaten Testsystem installieren. Entwicklungsbibliotheken und Quellbaum dürfen keine unbemerkte Laufzeitvoraussetzung sein.
3. Publisher und Desktop jeweils über den installierten Einstieg starten. Unter macOS `.app` öffnen, nicht eine Datei innerhalb des Bundles als Shellskript behandeln.
4. Neue Testdatenbank und Testadministrator verwenden. Anmeldung, Personenanlage und Programmneustart prüfen.
5. Mit passender Testlizenz die freigeschalteten Module und deren Abhängigkeiten prüfen. Standalone sowie aktivierungspflichtigen Ablauf getrennt testen, wenn beide angeboten werden.
6. Serverpaket im Vordergrund starten, Clientverbindung und kontrolliertes Beenden testen. Automatischen Dienststart gesondert einrichten und prüfen.
7. Upgrade mit gesichertem Testdatenbestand prüfen. Eine Neuinstallation ist kein Datenbankbackup und kein Migrations-Rollback.
8. Paket erst nach erfolgreicher Prüfung weitergeben. Codesignierung/Notarisierung bei macOS und Authenticode bei Windows sind im aktuellen Buildskript noch nicht automatisiert.

Bei einem mit Public-Key gebauten Kundenpaket führt der Treiber die nativen Tests in einem separaten ungebundenen `-verification`-Build aus. Anschließend entsteht der Produktionsbuild mit festem Vertrauensanker. Der Testbuild gehört nicht in das Kundenpaket. Qt-Runtime-Deployment wird beim Paketieren aktiviert, soweit das SDK es unterstützt; insbesondere Linux- und Archivpakete müssen auf ihre externen Bibliotheksabhängigkeiten geprüft werden.

Für die Build-/Paketierungslogik zusätzlich:

```
python3 tests/build_scripts_tests.py
```

Auf macOS erzeugt dieser Test auch ein echtes synthetisches Paket und entpackt es zur Strukturprüfung. Er installiert keine Anwendung und ersetzt keinen GUI-Installationstest. Native C++-/Qt-Tests sind nur mit vollständig eingerichteter Buildumgebung aussagekräftig.

11. Fehlerbilder und gezielte Behebung

Beobachtung	Prüfung und nächster Schritt
No such file ... <code>public.hex</code>	Tatsächliche vorhandene Datei auswählen; keine Platzhalterpfade kopieren
Nur <code>_CPack_Packages</code> vorhanden	Buildlog bis zum ersten Fehler lesen; noch kein fertiger Installer
CPack meldet README/Lizenz-Endung auf macOS	Aktuelle direkte pkgbuild/productbuild-Paketierung verwenden
Paket möchte auf Systemvolume schreiben	Aktuellen Paketcode verwenden; Ziel muss <code>/Applications/ClubPlatform...</code> sein
<code>._bin</code> , <code>bin/. _tool</code> in Payloadprüfung	Aktuelle AppleDouble-Prüfung verwenden; nicht beliebige unerwartete Pfade freischalten
Finder startet falsche oder alte App	Alte Prozesse beenden, installierten vollständigen <code>.app</code> -Pfad öffnen
Alte Lizenz ohne Installationsbindung	Beide Programme aktualisieren und ausdrücklich neue Standalone- oder gebundene Lizenz ausstellen
Keine Berechtigung	Angemeldeter Benutzer benötigt <code>schema.manage</code> ; Lizenz und Benutzerrechte getrennt prüfen
Lizenzsignatur ungültig	Herausgeber-Public-Key im Kundenbuild mit ausgewähltem Signierschlüssel vergleichen; JSON unverändert lassen
Modul nicht lizenziert	Modul-ID und Abhängigkeiten in Lizenz prüfen; bei gebundener Lizenz Aktivierung abschließen
Qt nicht gefunden / falsche Architektur	<code>QT_ROOT</code> , CMake-Cache, Compiler und Abhängigkeiten auf denselben SDK-/Architekturstand bringen
Fehler nach Generatorwechsel	Eigenen frischen Buildbaum verwenden; widersprüchlichen CMake-Cache nicht weiterverwenden

Buildfehler nicht durch `sudo` für den gesamten Build umgehen. Administratorrechte sind gegebenenfalls zur Installation nötig, nicht zur normalen Kompilierung im eigenen Projektverzeichnis. Bei verbleibenden Problemen den ersten Fehler, den vollständigen verwendeten Befehl und den Quellcommit festhalten.

12. Quellen und Pflege

Maßgeblich für diese Anleitung sind im privaten Repository `scripts/build.py`, die vier Shell-/PowerShell-Wrapper, `CMakeLists.txt`, `CMakePresets.json`, `vcpkg.json`, `tools/publisher`, `tools/activation` sowie die CMake-Dateien von Desktop und Server. Die kompakte Buildreferenz bleibt als Schnelleinstieg verfügbar.

Die PDF-Fassung wird aus dieser Markdown-Datei erzeugt. Bei Änderungen an Schaltern, Installationspfaden oder Lizenzrichtlinien zuerst Markdown aktualisieren und anschließend beide PDFs mit `scripts/render-guides.py` neu erzeugen. Die Generatoranleitung liegt in `docs/pdf/README.md`.